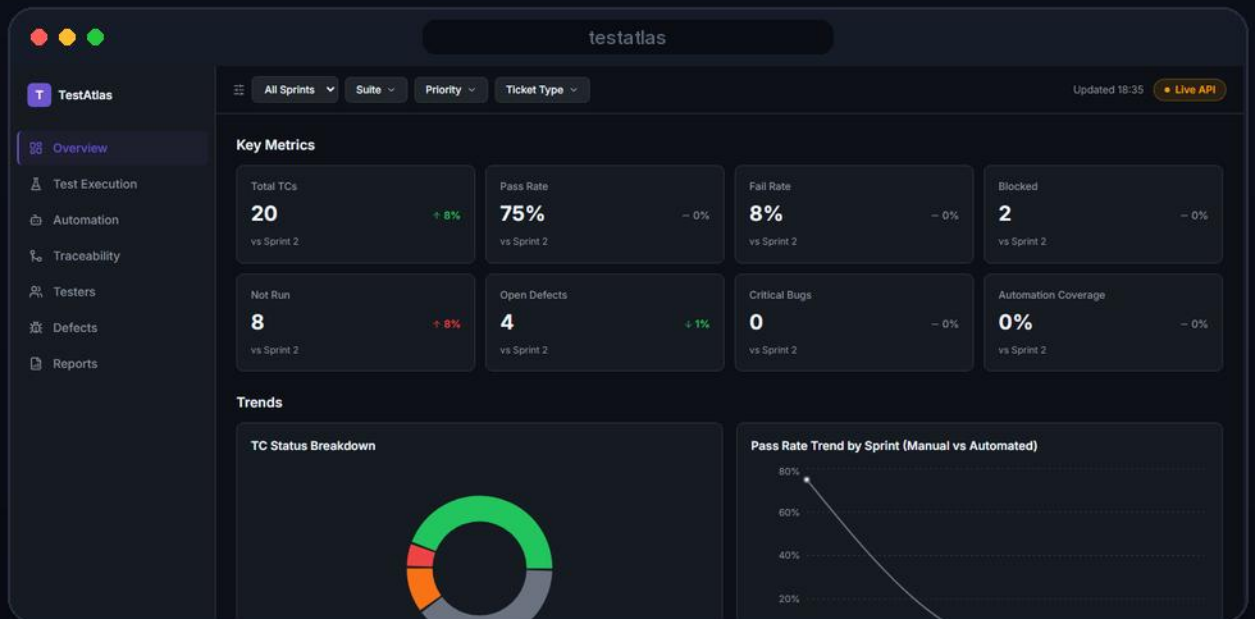




I SHIPPED A QA TOOL

One dashboard. Any defect tracker. Any test tool.



TestAtlas unifies your defect tracker + test case manager into one live view — KPIs, execution trends, automation coverage, traceability, tester analytics, and sprint reports.



THE PROBLEM

Every QA team runs the same ritual.

- 01 Open Jira for bugs
- 02 Open TestLink for test cases
- 03 Open a spreadsheet to reconcile them
- 04 Try to answer: are we shipping quality?

TestAtlas replaces step 3 and 4.

12

integrations

7

tabs

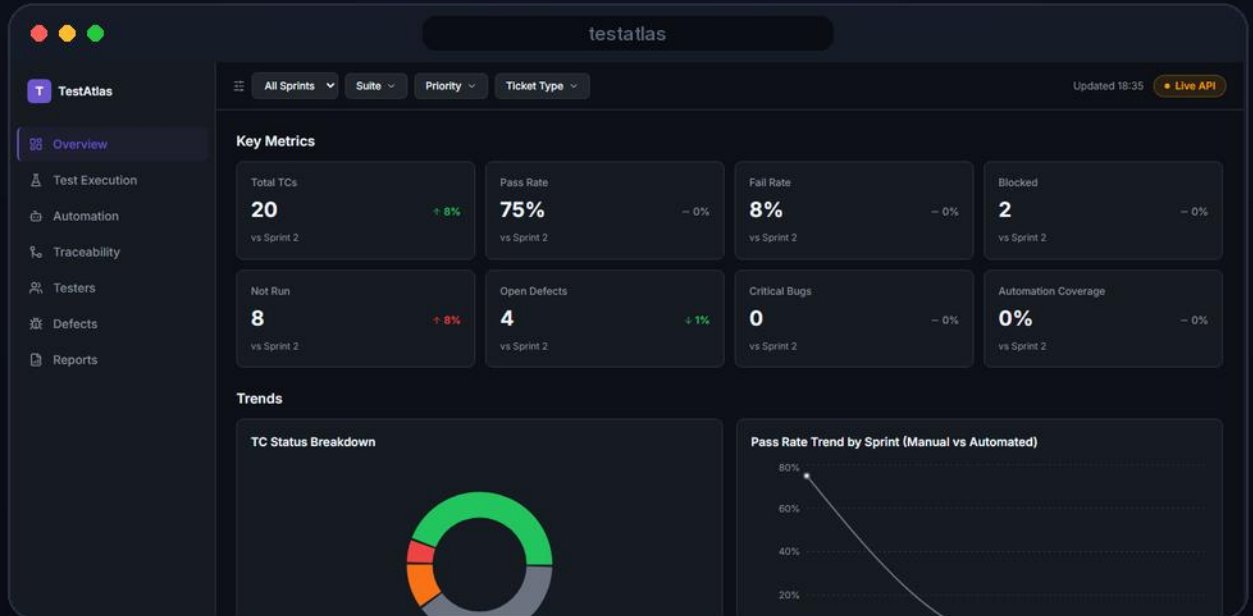
0

extra tools needed



OVERVIEW

Every KPI above the fold.
Nothing hidden.

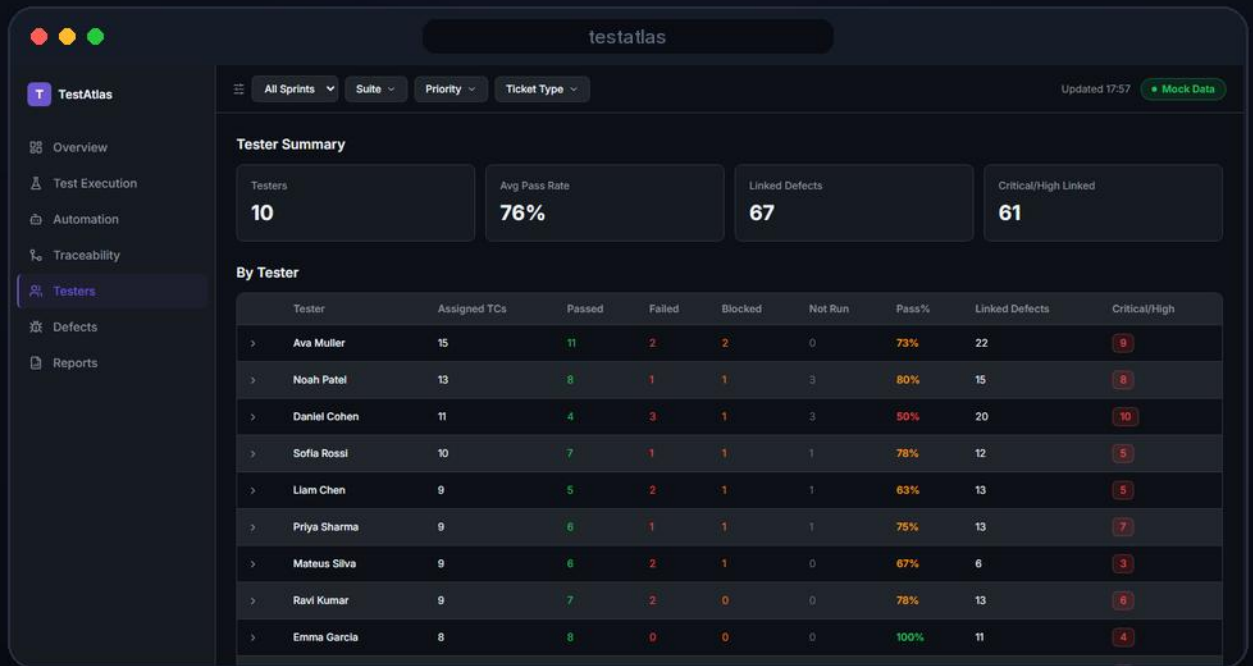


- Pass Rate · Fail Rate · Blocked · Automation Coverage — with sprint-over-sprint delta
- Attention callouts: suites below 60%, uncovered ticket types, failing automated TCs



TESTERS

The tab no QA dashboard ships.



10 testers ranked by pass%, assigned TCs, fails, blocks, linked defects, and Critical/High count — per sprint, per suite, filterable.



AUTOMATION

Coverage by suite, by ticket type, by sprint.



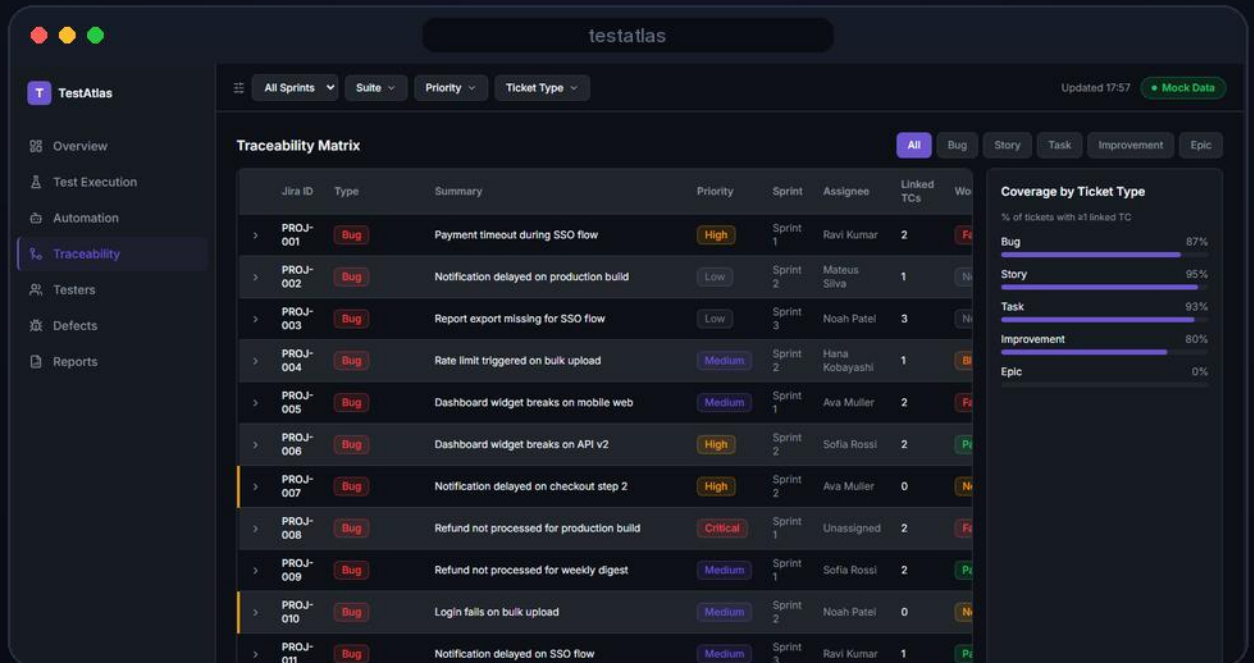
→ 53% coverage · 85% automated pass rate · +11.7% coverage delta vs last sprint

→ Automation gap table: which manual TCs haven't been automated yet



TRACEABILITY

Every ticket linked to its test cases. Gaps flagged.



Ticket ↔ test case matrix with linked-TC count, worst status, coverage % by ticket type — uncovered tickets highlighted.

PROOF

6 tool combos. One adapter each.



● Jira + TestLink



● Azure + TestRail



● Bugzilla + qTest



● Mantis + Zephyr



● GitHub + PractiTest



● Linear + Kiwi TCMS

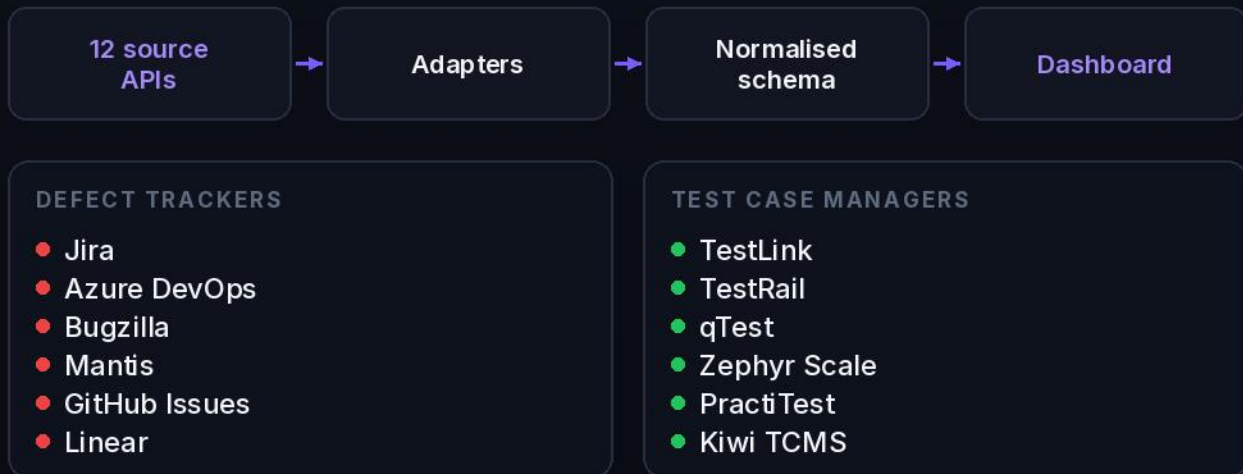
Jira + TestLink: verified against live accounts · other 10: verified end-to-end against mocked APIs



ARCHITECTURE

One schema. Twelve integrations.

The dashboard never talks to Jira or TestLink directly. It only reads one normalised Ticket/TestCase schema. Each tool is a single adapter that maps its API onto that contract — switching source is a config line, not a rewrite.





WHAT'S INSIDE

7 tabs. Each one a decision.

Overview — KPIs + sprint deltas + attention callouts

Test Execution — Suite health per sprint, daily trend, manual vs auto

Automation — Coverage by suite/ticket type, trend, automation gap

Traceability — Ticket ↔ TC matrix, coverage gaps surfaced

Testers — Per-tester pass%, fails, linked defects, C/H count

Defects — Bug trends by priority/status, open vs closed trend

Reports — Sprint summary, top failures, CSV/PDF export



WANT IT?

MIT licensed. Runs on mock data out of the box.

1 git clone the repo (link in comments)

2 npm install && npm run dev

3 Switch to your real Jira + TestLink

4 Write one adapter for any other tool

How does your team track quality today?

Curious what's working — and what's still stuck in a spreadsheet.